

DANH SÁCH BÁO CÁO LỖI HỆ THỐNG SUT (BUG REPORTS FOR ESHOP-SUT)

Dự án kiểm thử: HW05 – Performance Testing

Sinh viên: BaoBeiii – MSSV: 23127327

Đối tượng: eshop-sut/backend/server.js

Mục tiêu: Báo cáo các lỗi nghiệp vụ, bảo mật và hiệu năng phát hiện được trong quá trình rà soát mã nguồn SUT và thực thi kịch bản đo tải.



Tổng Hợp Các Lỗi Phát Hiện Được

Mã Bug	Tên Lỗi / Vấn Đề	Mức Độ	Endpoint Ảnh Hưởng	Trạng Thái
BUG-01	Công thức tính giảm giá phần trăm bị sai logic làm đội giá đơn hàng	Critical	POST /api/apply-coupon	Đã bắt lỗi qua k6 assertion
BUG-02	Lỗi hỏng SQL Injection & Crash HTML 500 khi tìm kiếm sản phẩm	High	GET /api/products?search=	Đã bắt lỗi qua k6 assertion
BUG-03	Sai lệch kiểu dữ liệu giá sản phẩm (trả về String thay vì Number ở ID chắn)	Medium	GET /api/products/:id	Đã bắt lỗi qua k6 assertion
BUG-04	Cơ chế Account Lockout (FR-02) cộng dồn sai số lần thử và khóa quá lâu	High	POST /api/login	Đã bắt lỗi qua k6 Stress test
BUG-05	Thanh toán đơn hàng không kiểm tra lại giỏ hàng và không tính lại tổng tiền	High	POST /api/checkout	Vi phạm đặc tả FR-08
BUG-06	Máy trạng thái đơn hàng cho phép chuyển từ 'canceled' sang 'delivered'	Medium	PUT /api/admin/orders/:id/status	Vi phạm đặc tả FR-10
BUG-07	Race condition khi dùng coupon dưới tải đồng thời (thiếu DB transaction)	Critical	POST /api/apply-coupon & /api/coupon-usage	Gây bypass giới hạn mã

Chi Tiết Các Bug & Bản Soạn Thảo GitHub Issue

● BUG-01: Sai logic tính giảm giá phần trăm (`POST /api/apply-coupon`)

- Vị trí mã nguồn: `backend/server.js:398-401` và `backend/server.js:419-421`
- Mô tả: Đoạn code tính giảm giá phần trăm đang dùng công thức:

```
discount_amount = Math.floor(total_amount * (1 - coupon.discount_value));
const final_amount = total_amount - discount_amount;
```

Nếu đơn hàng có `total_amount = 200,000` và mã giảm giá 10% (`discount_value = 10`):

- Giá trị giảm tính ra: $200000 \times (1 - 10) = 200000 \times (-9) = -1,800,000$ \$ đ!
 - Số tiền thanh toán cuối cùng: $200000 - (-1800000) = 2,000,000$ \$ đ (tăng gấp 10 lần giá trị đơn thay vì giảm 10%)!
- Kịch bản kiểm thử bắt lỗi: `scripts/23127327_Load_20260902.js` với assertion `r.json('final_amount') < totalAmount && r.json('discount_amount') > 0`.
 - Đề xuất khắc phục:

```
- discount_amount = Math.floor(total_amount * (1 - coupon.discount_value));
+ discount_amount = Math.floor(total_amount * (coupon.discount_value / 100));
```

● BUG-02: Lỗi hỏng SQL Injection & Crash 500 khi tìm kiếm sản phẩm (`GET /api/products`)

- Vị trí mã nguồn: `backend/server.js:144`
- Mô tả: API ghép chuỗi trực tiếp tham số người dùng vào truy vấn SQL:

```
const query = `SELECT * FROM products WHERE name LIKE '%${searchQuery}%'`;
```

Khi query chứa dấu nháy đơn `'` hoặc ký tự đặc biệt, SQLite báo lỗi cú pháp và backend trả về chuỗi HTML `<h1>Database Error</h1>` với HTTP 500 thay vì JSON response.

- Đề xuất khắc phục: Sử dụng Parameterized Query của SQLite:

```
- const query = `SELECT * FROM products WHERE name LIKE '%${searchQuery}%'`;
- db.all(query, [], (err, rows) => { ... });
+ const query = `SELECT * FROM products WHERE name LIKE ?`;
+ db.all(query, [`%${searchQuery}%`], (err, rows) => { ... });
```

● BUG-03: Sai lệch kiểu dữ liệu giá tiền (GET /api/products/:id)

- Vị trí mã nguồn: `backend/server.js:162`
- Mô tả: Tồn tại dòng code cố tình ép kiểu giá sang String đối với các sản phẩm có ID chẵn:

```
if (row.id % 2 === 0) row.price = row.price.toString();
```

Điều này phá vỡ tính nhất quán của API contract, khiến frontend/client nhận được chuỗi `"350000"` thay vì số nguyên `350000`.

- Kịch bản kiểm thử bắt lỗi: Check assertion: `typeof r.json('price') === 'number'`.
- Đề xuất khắc phục: Xóa bỏ điều kiện ép kiểu ép buộc.

● BUG-04: Account Lockout tăng sai số lần thử và khóa quá hạn (POST /api/login)

- Vị trí mã nguồn: `backend/server.js:54-57`
- Mô tả: Đặc tả FR-02 quy định: Đăng nhập sai mỗi lần tăng 1; sai liên tiếp từ 3 lần trở lên thì khóa 30 giây. Tuy nhiên mã nguồn lại viết:

```
const newAttempts = user.login_attempts + 2; // Tăng 2 thay vì 1
if (newAttempts >= 3) {
    lockedUntil = new Date(Date.now() + 180000).toISOString(); // Khóa 180s (3 ph
}
```

Hậu quả: Chỉ cần người dùng nhập sai **lần thứ 2** là tài khoản đã bị khóa ngay lập tức trong suốt 3 phút!

- Kịch bản kiểm thử bắt lỗi: `scripts/23127327_Stress_20260902.js` kích hoạt và phát hiện mã HTTP 403 sớm bất thường.
- Đề xuất khắc phục:

```
- const newAttempts = user.login_attempts + 2;
+ const newAttempts = (user.login_attempts || 0) + 1;
  if (newAttempts >= 3) {
-     lockedUntil = new Date(Date.now() + 180000).toISOString();
+     lockedUntil = new Date(Date.now() + 300000).toISOString();
  }
```

● BUG-05: Checkout không xác thực lại giỏ hàng và tổng tiền (POST /api/checkout)

- Vị trí mã nguồn: `backend/server.js:296-308`
- Mô tả: Backend tin tưởng 100% tham số `total_amount` do client gửi lên trong payload JSON mà không hề truy vấn CSDL để đối chiếu giá niêm yết hiện hành của các sản phẩm hay tính toán lại số lượng, vi phạm trực tiếp đặc tả toàn vẹn nghiệp vụ FR-08. Kẻ tấn công có thể chỉnh sửa request client gửi lên `total_amount: 1` hoặc `0` cho một giỏ hàng trị giá hàng chục triệu đồng, và backend vẫn tạo đơn hàng thành công!
- Đề xuất khắc phục:

```
// Backend phải tự tính toán lại dựa trên ID sản phẩm và số lượng
const calculatedTotal = items.reduce((sum, item) => sum + (productMap[item.id].pr
if (calculatedTotal !== req.body.total_amount) {
    return res.status(400).json({ error: "Tampered order amount detected" });
}
```

● BUG-06: Máy trạng thái đơn hàng cho phép chuyển phi lý từ 'canceled' sang 'delivered' (PUT /api/admin/orders/:id/status)

- Vị trí mã nguồn: `backend/server.js:550-552`
- Đặc tả vi phạm: Vi phạm máy trạng thái đơn hàng (Order State Machine) theo đặc tả FR-10.
- Mô tả chi tiết: Đặc tả FR-10 quy định vòng đời chuyển trạng thái hợp lệ của đơn hàng EShop:
 - `$\text{pending} \rightarrow \text{confirmed} \rightarrow \text{shipping} \rightarrow \text{delivered}$`
 - `$\text{pending} / \text{confirmed} \rightarrow \text{canceled}$` Khi đơn hàng đã bị hủy (`canceled`), đây là **trạng thái kết thúc cuối cùng (Terminal State)**. Không thể tiếp tục giao hàng hay hoàn tất một đơn hàng đã bị hủy. Tuy nhiên, trong mã nguồn `backend/server.js` tại dòng 550-551 tồn tại đoạn mã ngoại lệ cố ý:

```
// backend/server.js: dòng 550-551
if (currentStatus === "canceled" && status === "delivered")
    isValidTransition = true;
```

Hậu quả: Admin hoặc kẻ có quyền có thể chuyển thẳng một đơn hàng đã hủy (`canceled`) thành đã giao hàng thành công (`delivered`) mà không hề qua các bước

xác nhận hay vận chuyển. Lỗi này gây sai lệch nghiêm trọng về tồn kho kho hàng, số liệu tài chính đối soát doanh thu, và gian lận hệ thống kế toán.

- **Kịch bản kiểm thử bắt lỗi:**

- i. Tạo đơn hàng và hủy đơn: `PUT /api/orders/:id/cancel` $\rightarrow$ trạng thái là `canceled`.

- ii. Gửi request cập nhật Admin:

```
PUT /api/admin/orders/:id/status
Content-Type: application/json
Authorization: Bearer <admin_token>

{ "status": "delivered" }
```

- iii. **Kết quả thực tế của SUT:** Trả về HTTP `200 OK` (`{"message": "Order status updated"}`), trạng thái đơn hàng trong database bị đổi thành `delivered`.

- iv. **Kết quả mong đợi (Expected):** Phải trả về HTTP `400 Bad Request` (`{"error": "Invalid state transition from canceled to delivered"}`).

- **Đề xuất khắc phục:** Xóa bỏ hoàn toàn điều kiện cho phép chuyển trạng thái phi lý này:

```
--- a/eshop-sut/backend/server.js
+++ b/eshop-sut/backend/server.js
@@ -547,9 +547,6 @@ app.put("/api/admin/orders/:id/status", authenticateToken, (r
     if (currentStatus === "shipping" && status === "delivered")
         isValidTransition = true;

-     if (currentStatus === "canceled" && status === "delivered")
-         isValidTransition = true;
-
     if (!isValidTransition) {
         return res.status(400).json({
             error: `Invalid state transition from ${currentStatus} to ${status}`,
```

● **BUG-07: Race condition khi áp dụng Coupon dưới tải đồng thời (POST /api/apply-coupon & /api/coupon-usage)**

- **Vị trí mã nguồn:** `backend/server.js:387-395` và `backend/server.js:444-454`
- **Mô tả:** Việc kiểm tra số lần sử dụng coupon (`SELECT COUNT(*) FROM coupon_usage WHERE coupon_id = ? AND user_id = ?`) và việc ghi nhận lượt sử dụng (`INSERT INTO coupon_usage`) là 2 lời gọi hàm bất đồng bộ tách rời, không được bảo vệ bởi Database Transaction hay Mutex Lock. Khi có nhiều request đồng thời gửi lên cùng lúc (ví dụ

trong kịch bản Spike Test 150 VUs hoặc Stress Test 200 VUs), tất cả các luồng cùng đọc ra `usage_count = 0` trước khi bất kỳ luồng nào kịp `INSERT`, cho phép một người dùng áp dụng mã giảm giá nhiều lần vượt quá giới hạn `max_uses_per_user`.

- **Đề xuất khắc phục:** Sử dụng Transaction nguyên tử với SQLite `BEGIN IMMEDIATE TRANSACTION` hoặc `db.serialize()`:

```
db.serialize(() => {
  db.run("BEGIN IMMEDIATE TRANSACTION;");
  // Thực hiện SELECT COUNT(*) và INSERT trong cùng một transaction độc quyền
  db.run("COMMIT;");
});
```